

Cross-Residual Graph Neural Networks for Biomolecular Graph Classification: An Empirical Study of Residual Routing

Xuelin Hu¹ and Xiaoqin Fu²

¹School of Railway Communication and Signaling Engineering, Liuzhou Railway Vocational Technical College, Liuzhou 545000, China

²School of Railway Locomotive and Rolling Stock, Liuzhou Railway Vocational Technical College, Liuzhou 545000, China

Corresponding author:
Xiaoqin Fu²

Email address: fuxiaoqin@ltzy.edu.cn

ABSTRACT

Background. Graph neural networks are widely used for graph classification on proteins and molecules, but deeper message passing can blur discriminative local structure, and standard skip connections do not clarify whether cross-branch historical exchange offers a better trade-off than ordinary residual reuse. This issue is especially relevant in biomolecular graph classification, where useful evidence may depend on both local motifs and graph-level context.

Methods. We study Cross-Residual Graph Neural Networks (CR-GNN) as a controlled comparison framework that evaluates plain stacking, node-level residual reuse, node-level cross-residual exchange, graph-level residual propagation, and graph-level cross-summary exchange under shared graph-classification backbones. The main benchmark uses six datasets, nine methods, four message-passing operators, and stratified five-fold evaluation with mean best test accuracy and fold-level standard deviation. We further organize targeted experiments on gate control, residual routing modes, residual-parameter sweeps, and exploratory sensitivity scans.

Results. The five-fold results show that residual reuse remains the stronger default family across the biomolecular benchmark package, while cross-residual routing usually improves over plain stacking and yields selective gains on specific datasets rather than a uniform advantage. Within the biological core, GraphResGNN is strongest on PROTEINS and ENZYMES, whereas NodeResGNN is strongest on DD. The targeted routing analyses further show that routing design materially affects performance: learnable dense routing remains strongest on the best graph-level residual line, whereas mild sparse routing is strongest on three of four representative routing targets.

Conclusions. CR-GNN is best interpreted as a controlled biomolecular graph-classification design space rather than a universally dominant architecture. The empirical evidence supports a bounded conclusion: cross-residual interaction is useful but conditional, residual reuse remains the stronger default reference, and routing strategy is an important determinant of performance.

INTRODUCTION

Graph neural networks (GNNs) are now a standard approach for learning from molecular and protein graphs because they encode local neighborhood structure together with higher-order relational context (Kipf and Welling, 2016; Gilmer et al., 2017; Hamilton et al., 2017; Wu et al., 2021; Zhou et al., 2020). In biomolecular graph classification, this combination matters because predictive evidence may depend simultaneously on local biochemical motifs and broader graph-level organization. Benchmark suites such as PROTEINS, DD, ENZYMES, MUTAG, AIDS, and Mutagenicity therefore remain useful as controlled testbeds for methodology-oriented graph classification studies rather than as direct proxies for downstream biological discovery.

The main methodological difficulty is that deeper message passing is not automatically beneficial in these settings. Repeated propagation can suppress informative intermediate states

through over-smoothing and related depth effects (Li et al., 2018; Oono and Suzuki, 2020; Chen et al., 2020b), while over-squashing can cause information from distant nodes to be compressed into uninformative bottlenecks (Alon and Yahav, 2021; Topping et al., 2021). Residual connections, normalization, and regularization alleviate some of this degradation (He et al., 2016; Bresson and Laurent, 2017; Zhao and Akoglu, 2020; Cai et al., 2021; Rong et al., 2019; Chen et al., 2020a), but most residual designs still reuse information along the same branch. As a result, they improve optimization without directly testing whether alternative information-flow topologies preserve more useful historical signals.

Related graph-learning work has already explored skip connections, dense aggregation, APPNP-style propagation, Jumping Knowledge readout, and hierarchical pooling (Xu et al., 2018; Klicpera et al., 2018; Gao and Ji, 2019; Ying et al., 2018). These methods preserve information from multiple depths or across pooling hierarchies, yet they do not directly isolate whether exchanging hidden states across branches or reinjecting pooled graph summaries across block boundaries is preferable to ordinary same-path residual reuse. This is the narrower gap addressed in the present paper.

This paper studies Cross-Residual Graph Neural Networks (CR-GNN), a controlled model family that compares plain stacking, node-level residual reuse, node-level cross-residual exchange, graph-level residual propagation, and graph-level cross-summary exchange under shared graph-classification backbones. The study is organized around three specific research questions:

1. When does cross-residual interaction help? Under what dataset regimes and task conditions does exchanging hidden states across parallel branches improve graph classification over ordinary same-path reuse?
2. When does ordinary residual reuse remain the stronger default? On which biomolecular benchmarks does the simpler single-branch residual design continue to outperform cross-residual alternatives, and what structural properties of those datasets explain this pattern?
3. How does residual routing design change the answer? Beyond the binary question of whether an extra reuse path exists, how do different filtering strategies—learnable dense routing, top-k selection, and sparse suppression—affect the relative performance of residual and cross-residual architectures?

Framing the study around these three questions is important for the paper’s methodological scope. Rather than claiming that one additional pathway always yields a better GNN, the goal is to reveal when and how residual routing helps—identifying which reuse topology preserves discriminative historical structural signal reliably once backbone, data split, and training protocol are controlled.

The work is organized as a biomolecular graph-classification methods study with a mechanism-oriented empirical focus. The main evidence comes from a protein-oriented benchmark core built around PROTEINS, DD, and ENZYMES, together with supplementary molecular robustness datasets. This design keeps the paper close to biologically meaningful graph settings while maintaining a reproducible and controlled benchmark scope. The later sections therefore move from the architectural motivation to the common methodological setting, then to the confirmatory benchmark results, targeted ablations, and supporting sensitivity analyses. This progression is intentional: the benchmark first establishes the empirical pattern, and the gate and routing studies then test whether that pattern can be explained by how residual information is passed forward.

Contributions

The main contributions of this work are:

- A controlled comparison framework for reuse topologies. Rather than proposing a single new architecture and claiming it is universally better, this work compares five information-flow designs—plain stacking, node-level residual reuse, node-level cross-residual exchange, graph-level residual propagation, and graph-level cross-summary exchange—under identical backbones, data splits, and training protocols. This controlled setup isolates the effect

of reuse topology from confounding factors such as model capacity, operator choice, or training hyperparameters, and makes it possible to ask when a given reuse strategy helps rather than whether it wins on average.

- Empirical evidence for conditional, rather than universal, gains. Under stratified five-fold evaluation across six biomolecular datasets, residual reuse (NodeResGNN, GraphResGNN) remains the most reliable default choice, but the margin over cross-residual variants is modest and not uniform. Cross-residual routing provides clear improvements over plain stacking on most datasets and becomes the top-ranked design on selected molecular benchmarks such as MUTAG and Mutagenicity. The overall pattern is one of selective benefit rather than universal dominance: the value of an additional cross-branch pathway depends on dataset characteristics, and no single architecture ranks first everywhere.
- A routing-level mechanism analysis. Going beyond architecture labels, this study examines how residual information is filtered before reuse—comparing learnable dense routing, top-k channel selection, and sparse suppression under the same five-fold protocol. The results show that routing strategy is target-dependent: learnable dense routing remains strongest on the best graph-level residual line, while mild sparse routing is often advantageous on cross-oriented targets. This finding reframes the discussion from “should we add a residual path?” to “how should historical information be filtered when it is passed forward?”
- A practical distinction between node-level and graph-level reuse. The location of information reuse matters at least as much as whether reuse exists. Node-level cross exchange (NodeCrossGNN) is the more stable cross-oriented design across datasets, whereas graph-level cross routing (GraphCrossGNN) is more regime-sensitive and, under several operator–dataset combinations, is statistically significantly worse than the corresponding graph-level residual baseline. This distinction provides practical guidance for practitioners choosing between reuse strategies.

MATERIALS AND METHODS

This section defines the common graph-classification setting, the CR-GNN architecture family, the benchmark package, and the shared experimental protocol. The guiding principle is to isolate where information is reused while keeping the remaining pipeline as consistent as possible. That design choice is central to the paper’s methodological justification: if backbone operator, pooling, and evaluation protocol are held fixed, then performance differences are more plausibly attributable to the reuse topology itself rather than to unrelated implementation changes.

Problem definition and notation

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ denote an undirected or directed graph, where $\mathcal{V} = \{v_1, v_2, \dots, v_n\}$ is the set of n nodes and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of edges. Each node $v_i \in \mathcal{V}$ is associated with a feature vector $\mathbf{x}_i \in \mathbb{R}^{d_{\text{in}}}$, and the node features for the entire graph are collected in $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$. The graph structure is represented using an adjacency matrix $\mathbf{A}$.

Given a training dataset $\mathcal{D} = \{(\mathcal{G}_i, y_i)\}_{i=1}^N$, the objective is to learn a mapping function $f_{\theta} : \mathcal{G} \rightarrow \hat{y}$ that predicts the label of an unseen graph. The present study compares five architectures under the same graph-classification pipeline:

- PlainGNN: sequential message passing without explicit state reuse
- NodeResGNN: single-branch node-level residual reuse
- NodeCrossGNN: two-branch node-level cross exchange
- GraphResGNN: graph-level residual propagation across sequential blocks
- GraphCrossGNN: graph-level cross-summary exchange across paired blocks

143 The comparison is designed to isolate where information is reused: inside one branch, across
 144 parallel branches, or through pooled graph summaries passed between blocks. This isolates a
 145 concrete research question behind the architecture family. Same-branch residual reuse mainly
 146 tests whether re-injecting historical states stabilizes deeper propagation, whereas cross-residual
 147 designs test whether exchanging information across parallel streams can preserve complementary
 148 structure that would otherwise be lost under ordinary stacking.

149 Shared message passing and graph-level learning

150 Let Φ denote a message-passing operator. For a chosen backbone, node representations are
 151 updated over L layers as

$$\mathbf{h}_i^{(\ell+1)} = \sigma \left(\mathbf{W}^{(\ell)} \cdot \text{AGGREGATE}_{\Phi} \left(\left\{ \mathbf{h}_j^{(\ell)} : j \in \mathcal{N}(i) \right\}, \mathbf{h}_i^{(\ell)} \right) + \mathbf{b}^{(\ell)} \right). \quad (1)$$

152 After L layers, a readout function R aggregates node-level representations into a graph embedding

153

$$\mathbf{h}_{\mathcal{G}} = R \left(\left\{ \mathbf{h}_i^{(L)} : i = 1, \dots, n \right\} \right). \quad (2)$$

154 The classifier produces

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}_{\text{clf}} \mathbf{h}_{\mathcal{G}} + \mathbf{b}_{\text{clf}}), \quad (3)$$

155 and the model is trained end-to-end by minimizing cross-entropy:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c}). \quad (4)$$

156 CR-GNN architecture family

157 CR-GNN is defined as a compact family of graph classification architectures. All variants
 158 share the same input projection, stacked message-passing layers, graph-level pooling, and linear
 159 classifier; they differ only in how intermediate node states and graph-level summaries are reused
 160 across depth and across branches. The design deliberately avoids introducing separate encoders,
 161 task-specific feature engineering, or architecture-specific readout heads, because such additions
 162 would make it harder to interpret whether any observed gain comes from residual topology or
 163 from extra model capacity.

164 Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a graph with node feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$ and adjacency structure $\mathbf{A}$.
 165 The model can be instantiated with one of four standard message-passing operators,

$$\Phi \in \{\text{GCNConv}, \text{GATConv}, \text{SAGEConv}, \text{GINConv}\},$$

166 and reuses the same operator type throughout a model instance. For a hidden width d , every
 167 branch starts with an input projection

$$\mathbf{H}^{(0)} = \text{ReLU}(\Phi_{\text{in}}(\mathbf{X}, \mathbf{A})), \quad (5)$$

168 followed by L hidden layers and a readout

$$\mathbf{g} = \text{Pool}(\mathbf{H}^{(L)}). \quad (6)$$

169 Single-branch and node-level residual variants

170 PlainGNN. The plain baseline stacks L copies of the same operator without residual reuse:

$$\mathbf{H}^{(\ell+1)} = \psi \left(\text{ReLU} \left(\Phi_{\ell}(\mathbf{H}^{(\ell)}, \mathbf{A}) \right) \right). \quad (7)$$

171 NodeResGNN. The first residual variant stores the previous hidden state within the same
 172 branch:

$$\mathbf{H}^{(\ell+1)} = \psi \left(\text{ReLU} \left(\Phi_{\ell}(\mathbf{H}^{(\ell)} + \mathbf{H}^{(\ell-1)}, \mathbf{A}) \right) \right). \quad (8)$$

NodeCrossGNN. The node-level cross-residual model maintains two parallel branches with separate input projections:

$$\mathbf{H}_1^{(\ell+1)} = \psi\left(\text{ReLU}\left(\Phi_\ell^{(1)}\left(\mathbf{H}_1^{(\ell)} + \tilde{\mathbf{H}}_2^{(\ell)}, \mathbf{A}\right)\right)\right), \quad (9)$$

$$\mathbf{H}_2^{(\ell+1)} = \psi\left(\text{ReLU}\left(\Phi_\ell^{(2)}\left(\mathbf{H}_2^{(\ell)} + \tilde{\mathbf{H}}_1^{(\ell)}, \mathbf{A}\right)\right)\right). \quad (10)$$

The final node representation is the branch sum

$$\mathbf{H}^{(L)} = \mathbf{H}_1^{(L)} + \mathbf{H}_2^{(L)}. \quad (11)$$

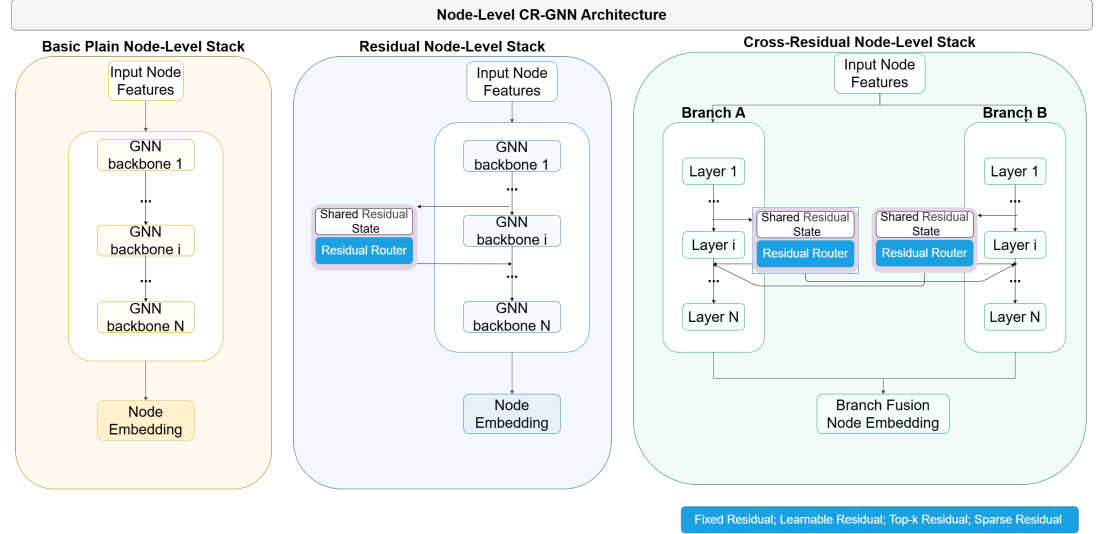


Figure 1. Node-Level CR-GNN Architecture. The three columns compare plain stacking (left), same-branch residual reuse (middle), and two-branch cross-residual exchange (right). In the cross-residual design, each branch reuses not only its own previous state but also the counterpart branch’s state from the previous layer, with residual routers controlling the strength and sparsity of cross-branch injection.

Graph-level residual propagation

GraphResGNN. This variant chains three graph-conditioned blocks:

$$\mathbf{g}^{(t+1)} = B_{t+1}\left(\mathbf{X}, \mathbf{A}, \mathbf{g}^{(t)}\right), \quad t \in \{1, 2\}. \quad (12)$$

GraphCrossGNN. Four blocks are organized as two sequential pairs. Within each stage, the two block outputs are swapped and reused as the next stage’s graph-level input:

$$(\mathbf{g}_1^{(t)}, \mathbf{g}_2^{(t)}) = \left(B_{2t-1}\left(\mathbf{X}, \mathbf{A}, \mathbf{g}_1^{(t-1)}\right), B_{2t}\left(\mathbf{X}, \mathbf{A}, \mathbf{g}_2^{(t-1)}\right)\right), \quad (13)$$

$$(\mathbf{g}_1^{(t+1)}, \mathbf{g}_2^{(t+1)}) = \left(\mathbf{g}_2^{(t)}, \mathbf{g}_1^{(t)}\right). \quad (14)$$

The final graph embedding is

$$\mathbf{g}_{\text{final}} = \mathbf{g}_1^{(T)} + \mathbf{g}_2^{(T)}. \quad (15)$$

Residual routing mechanisms

Beyond the architecture family itself, this study evaluates how residual information is injected. This second layer is necessary because the architecture comparison alone cannot answer whether an apparent residual advantage comes from the existence of an extra path or from the way that path is filtered. Three routing modes are used in the targeted ablations:

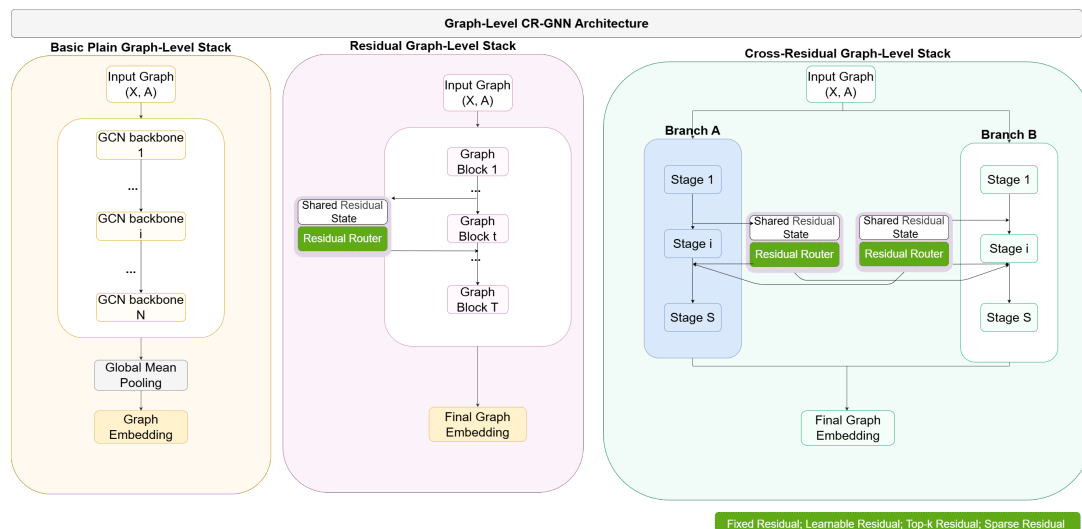


Figure 2. Graph-Level CR-GNN Architecture. The three columns compare plain graph-level stacking (left), sequential graph-level residual propagation (middle), and cross-summary exchange across paired blocks (right). In the cross-residual design, shared residual states and residual routers control graph-level information exchange between parallel block sequences, with four routing modes available: fixed, learnable, top-k, and sparse residual.

- Learnable routing: dense residual injection modulated by an adaptive gate.
- Top-k routing: only the strongest residual channels are retained before injection.
- Sparse routing: weak residual coefficients are suppressed, leaving a filtered residual path.

These routing modes are not used to redefine the core architecture family. Instead, they form a mechanism layer on top of selected residual and cross-residual targets in the experimental design. The routing study is therefore interpretive rather than decorative: it tests whether the same architecture behaves differently when residual information is passed forward densely, selectively, or sparsely.

Datasets

To test the proposed architecture in a biologically meaningful setting, the benchmarks are organized into a unified two-layer package.

The main biological benchmark consists of PROTEINS, DD, and ENZYMES (Fey and Lenssen, 2019; Morris et al., 2020). The supplementary robustness package consists of MUTAG, AIDS, and Mutagenicity. This organization keeps the main biological claim centered on protein-oriented datasets while still testing structural robustness on additional molecular graphs.

PROTEINS PROTEINS contains graph representations of proteins, where nodes denote secondary structure elements and edges encode neighborhood relations between them. The task is binary graph classification. The dataset contains 1,113 graphs, 2 classes, and 3 input features per node.

DD DD is a protein structure dataset in which each graph corresponds to a protein and nodes represent amino acids or structure-related units linked by proximity-derived edges. The task is binary classification. DD contains 1,178 graphs, 2 classes, and 89 input features.

ENZYMES ENZYMES is a 6-class enzyme classification benchmark with 600 graphs and 3 input features. It preserves an enzyme-oriented biological interpretation while remaining directly comparable to the other graph classification benchmarks in the study.

All active datasets are used through a unified benchmark interface (Fey and Lenssen, 2019; Morris et al., 2020). We keep preprocessing intentionally light and do not introduce task-specific feature engineering, graph augmentation, or edge redesign in the final protocol.

Table 1. Unified summary of the main biological benchmark package.

Dataset	Split	#Graphs	#Cls.	Feat.	Avg. N / E	Role
PROTEINS	5-fold CV + val	1113	2	3	39.1 / 72.8	main
DD	5-fold CV + val	1178	2	89	284.3 / 715.7	main
ENZYMES	5-fold CV + val	600	6	3	32.6 / 62.1	main

Table 2. Supplementary robustness datasets retained outside the core biological narrative.

Dataset	Split	#Graphs	#Cls.	Feat.	Avg. N / E	Role
MUTAG	5-fold CV + val	188	2	7	17.9 / 19.8	supp.
AIDS	5-fold CV + val	2000	2	38	15.7 / 16.2	supp.
Mutagenicity	5-fold CV + val	4337	2	14	30.0 / 30.6	supp.

214 All reported results follow the same stratified two-stage evaluation design:

- 215 • Outer evaluation: stratified 5-fold cross-validation at the graph level
- 216 • Inner validation: a stratified validation subset sampled from the training fold with valida-
217 tion ratio 0.1

218 The primary evaluation metric is graph classification accuracy on the held-out test fold. For
219 fold k with test set $\mathcal{T}_k$, the fold accuracy is

$$\text{Acc}_k = \frac{1}{|\mathcal{T}_k|} \sum_{(\mathcal{G}_i, y_i) \in \mathcal{T}_k} \mathbf{1}[\hat{y}_i = y_i]. \quad (16)$$

220 Across the five folds, we report the mean accuracy

$$\overline{\text{Acc}} = \frac{1}{5} \sum_{k=1}^5 \text{Acc}_k \quad (17)$$

221 and the corresponding standard deviation

$$\sigma_{\text{Acc}} = \sqrt{\frac{1}{5} \sum_{k=1}^5 (\text{Acc}_k - \overline{\text{Acc}})^2}. \quad (18)$$

222 Compared methods and backbone operators

223 The full paper-facing benchmark compares nine methods. The CR-GNN family contains five
224 models: PlainGNN, NodeResGNN, NodeCrossGNN, GraphResGNN, and GraphCrossGNN. Four
225 external baselines are also included in the benchmark package: GraphSAGEBaseline, GINBase-
226 line, JKNetBaseline, and APPNPBaseline. These baselines are intended to anchor the con-
227 trolled comparison against representative graph-classification families with distinct propagation
228 or aggregation behavior, rather than to claim exhaustive coverage of every recent leaderboard
229 method.

230 Each method is evaluated under four message-passing operators:

- 231 • GCNConv
- 232 • GATConv
- 233 • GINConv
- 234 • SAGEConv

235 Training configuration and implementation details

236 All models are implemented in PyTorch Geometric (Fey and Lenssen, 2019) and trained under a
237 unified protocol. The shared backbone across all CR-GNN variants and external baselines uses a
238 hidden dimension of $d = 64$, $L = 4$ message-passing layers, and a dropout rate of $p = 0.5$ applied
239 after each layer’s activation and before the final classifier. After the last message-passing layer,
240 node representations are aggregated via global mean pooling, and a single linear layer maps
241 the resulting graph embedding to class logits. No batch normalization or layer normalization is
242 applied; the only regularization is dropout and weight decay.

243 Training uses the Adam optimizer with an initial learning rate of 5×10^{-3} and weight decay
244 of 1×10^{-4} . The learning rate is reduced by a factor of 0.5 when validation loss fails to improve
245 for 15 epochs (ReduceLROnPlateau), with a minimum learning rate of 1×10^{-5} . Each run trains
246 for at most 200 epochs with a batch size of 256 graphs. Early stopping terminates training when
247 validation loss does not improve for 60 consecutive epochs. Gradients are clipped to a maximum
248 L_2 -norm of 2.0. The loss function is standard cross-entropy.

249 All train-validation-test splits are fixed by a global random seed of 1024, ensuring that every
250 model sees identical data partitions within each fold. No task-specific feature engineering, graph
251 augmentation, or edge redesign is applied; the raw node features and adjacency structures from
252 TUDataset are used directly.

253 The learnable gate used in residual and cross-residual variants applies a sigmoid activation
254 to a scalar parameter initialized at 0.8, allowing each residual injection to be adaptively scaled
255 during training. For the top-k routing mode, only the strongest fraction of residual channels
256 (determined by absolute magnitude) is retained before injection. For the sparse routing mode,
257 a softshrink operator with threshold λ suppresses residual coefficients whose magnitude falls
258 below λ , leaving a filtered residual path.

259 The four external baselines are configured as follows. GraphSAGEBaseline stacks SAGE-
260 Conv layers with mean aggregation. GINBaseline stacks GINConv layers, each containing a
261 two-layer MLP with ReLU activation. JKNetBaseline stacks GCNConv layers and aggregates
262 representations from all depths via Jumping Knowledge with concatenation mode (cat). APP-
263 NPBaseline uses a GCNConv backbone followed by APPNP propagation with $K = 10$ steps and
264 teleport probability $\alpha = 0.1$.

265 All experiments are run under a single version-controlled workflow. Each fold’s best check-
266 point is selected by minimum validation loss, and the corresponding test accuracy is recorded
267 as the fold result. The five fold-level test accuracies are then summarized as mean $\pm$ population
268 standard deviation.

269 Experimental design

270 The empirical study is organized to separate confirmatory five-fold evidence from exploratory
271 supporting analyses. The latest benchmark evaluates six datasets, nine methods, four message-
272 passing operators, and five outer folds under one unified protocol. This full matrix provides the
273 main comparison used throughout the paper, and the benchmark tables are placed directly in
274 the corresponding result subsections so that each empirical claim is supported in place.

275 The benchmark is divided into two layers. The biological core consists of PROTEINS, DD,
276 and ENZYMES, which anchor the main biomolecular interpretation of the paper. A supple-
277 mentary robustness package extends the analysis to MUTAG, AIDS, and Mutagenicity. This
278 split keeps the central claim focused while still testing whether the conclusions remain stable on
279 additional molecular graph settings.

280 Targeted ablations are organized to address the three research questions posed in the Intro-
281 duction through controlled mechanism experiments:

- 282 1. Q1: When does cross-residual interaction help? Cross-gate ablations. Six settings where
283 a cross architecture is already locally competitive are evaluated under four gate configu-
284 rations (learnable, fixed 0.0, fixed 0.5, fixed 1.0). This tests whether the observed cross-
285 residual advantage depends on adaptive injection control, or whether any fixed-weight ad-
286 ditional path produces similar gains. The comparison isolates the source of cross-residual
287 benefit: adaptive filtering versus structural pathway addition.

288 2. Q2: When does ordinary residual reuse remain the stronger default? Gate-control abla-
289 tions. On the strongest graph-level residual target (AIDS + GraphResGNN + GINConv),
290 learnable versus fixed-strength gates are compared under identical conditions. This di-
291 rectly tests whether the residual family’s default strength comes from adaptive reuse or
292 merely from having an extra injection path, and whether fixed residual injection can match
293 or surpass adaptive control.

294 3. Q3: How does residual routing design change the answer? Residual-routing ablations.
295 Four representative targets (spanning both residual-oriented and cross-oriented settings)
296 compare learnable dense routing, top-k channel selection at three retention ratios (0.25,
297 0.5, 0.75), and sparse suppression at three thresholds (0.02, 0.05, 0.10) under the same
298 five-fold protocol. This moves the analysis beyond architecture labels—whether a model is
299 called “residual” or “cross-residual”—to the mechanism level: how aggressively historical
300 information should be filtered before reuse.

301 Residual-parameter sweeps extend the routing analysis by varying top-k ratios and sparsity
302 strengths on the same representative targets. Older single-fold sensitivity scans over depth,
303 dropout, and learning rate are retained only as exploratory supporting analyses. In the final
304 interpretation, the five-fold latest-version runs are treated as the confirmatory evidence, while
305 the single-fold scans are used only to describe optimization trends rather than to support the
306 main claim.

307 Evaluation metrics and statistical reporting

308 The primary metric is graph classification accuracy on the held-out test fold. Main benchmark
309 and targeted ablation results are reported as five-fold mean accuracy with standard deviation.
310 In the final paper, the five-fold latest-version experiments are treated as the main confirma-
311 tory evidence, while older single-fold parameter scans are used only as supporting exploratory
312 analyses.

313 RESULTS

314 This section reports the final empirical evidence in four layers: the full benchmark, a mechanism-
315 oriented interpretation of the strongest residual patterns, the targeted gate and residual-routing
316 studies, and the exploratory sensitivity analyses. The evidence hierarchy is explicit. The main
317 benchmark and the targeted ablations use the latest stratified five-fold results and are reported
318 as mean best test accuracy $\pm$ fold-level standard deviation, whereas the older sensitivity scans
319 are retained only as supporting trend analyses. This separation matters because the paper’s
320 main claims are intended to rest on repeated-split evidence rather than on one favorable run.

321 Overall benchmark performance

322 The final benchmark evaluates six datasets, nine methods, four operators, and five outer folds
323 under one unified protocol. The main text therefore keeps the benchmark tables next to the
324 specific claims they support, rather than separating the tabular evidence into a standalone
325 appendix. This presentation is deliberate: the paper argues for a bounded empirical conclusion,
326 so the tabular evidence is placed where each claim is made and interpreted.

327 Taken together, the three tables show a consistent hierarchy. Residual reuse is still the
328 stronger default family in the protein-oriented core, cross-residual routing becomes selectively
329 useful on supplementary molecular datasets, and the winner summary confirms that no single
330 complexity trend explains the results by itself. The combination of mean values and standard
331 deviations is also important here: several gaps are modest rather than dramatic, which is exactly
332 why the paper interprets the results as a controlled pattern of relative stability rather than as
333 evidence of universal domination by one design.

Table 3. Main biological benchmark results. Cells show mean $\pm$ std [min, max] over five folds. Bold: best mean per dataset.

Model	PROTEINS	DD	ENZYMES
PlainGNN	0.7053 $\pm$ 0.0271 [0.667, 0.735]	0.7165 $\pm$ 0.0179 [0.694, 0.742]	0.2450 $\pm$ 0.0407 [0.183, 0.292]
NodeResGNN	0.6945 $\pm$ 0.0509 [0.599, 0.740]	0.7207 $\pm$ 0.0259 [0.689, 0.763]	0.2650 $\pm$ 0.0661 [0.158, 0.367]
NodeCrossGNN	0.6963 $\pm$ 0.0433 [0.626, 0.753]	0.7198 $\pm$ 0.0178 [0.706, 0.754]	0.3000 $\pm$ 0.0580 [0.233, 0.400]
GraphResGNN	0.7223 $\pm$ 0.0367 [0.653, 0.762]	0.6934 $\pm$ 0.0458 [0.609, 0.737]	0.3117 $\pm$ 0.0746 [0.167, 0.375]
GraphCrossGNN	0.6864 $\pm$ 0.0465 [0.595, 0.717]	0.7097 $\pm$ 0.0309 [0.668, 0.763]	0.3017 $\pm$ 0.0750 [0.183, 0.400]

Table 4. Supplementary robustness benchmark results. Cells show mean $\pm$ std [min, max] over five folds. Bold: best mean per dataset.

Model	MUTAG	AIDS	Mutagenicity
PlainGNN	0.7236 $\pm$ 0.0486 [0.676, 0.811]	0.8455 $\pm$ 0.0257 [0.800, 0.870]	0.7814 $\pm$ 0.0235 [0.749, 0.814]
NodeResGNN	0.7343 $\pm$ 0.0734 [0.649, 0.865]	0.8690 $\pm$ 0.0354 [0.800, 0.897]	0.7897 $\pm$ 0.0139 [0.766, 0.809]
NodeCrossGNN	0.7397 $\pm$ 0.0676 [0.676, 0.865]	0.8735 $\pm$ 0.0412 [0.795, 0.915]	0.8033 $\pm$ 0.0159 [0.776, 0.825]
GraphResGNN	0.7290 $\pm$ 0.0585 [0.676, 0.838]	0.9160 $\pm$ 0.0121 [0.892, 0.927]	0.8022 $\pm$ 0.0220 [0.770, 0.825]
GraphCrossGNN	0.7129 $\pm$ 0.0539 [0.649, 0.811]	0.8710 $\pm$ 0.0412 [0.800, 0.927]	0.8029 $\pm$ 0.0182 [0.774, 0.826]

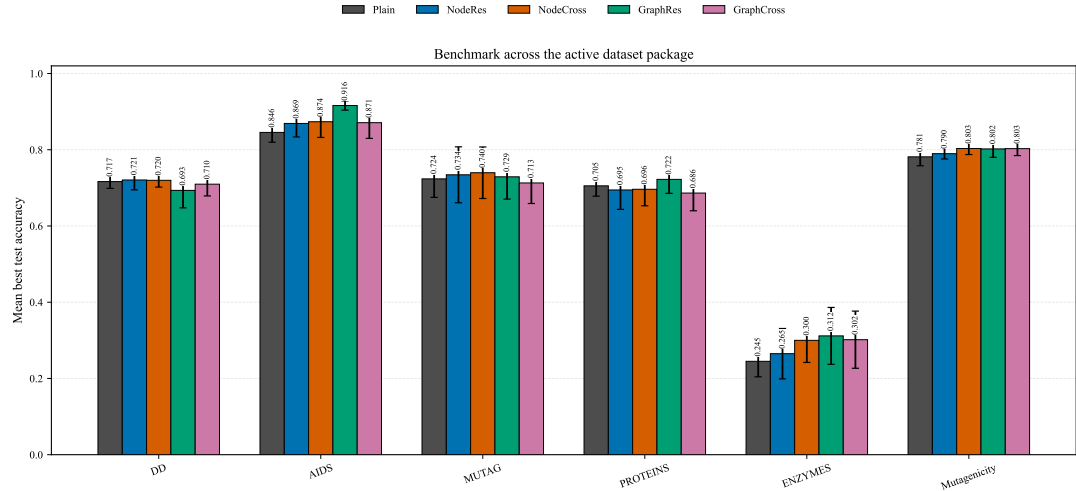


Figure 3. Benchmark over the active dataset package. Each bar reports mean best test accuracy, the numeric label above each bar gives the exact mean value, and error bars show the fold-level standard deviation.

Table 5. Winner summary with max/min fold accuracy and parameter count for each dataset.

Dataset	Winner	Mean Acc	Max Acc	Min Acc	Best Epoch	Params
PROTEINS	GraphResGNN	0.7223	0.7623	0.6532	58.8	51208
DD	NodeResGNN	0.7207	0.7627	0.6894	25.4	22530
ENZYMES	GraphResGNN	0.3117	0.3750	0.1667	91.4	52248
MUTAG	NodeCrossGNN	0.7397	0.8649	0.6757	88.2	34434
AIDS	GraphResGNN	0.9160	0.9275	0.8925	83.8	57928
Mutagenicity	NodeCrossGNN	0.8033	0.8247	0.7756	131.2	35330

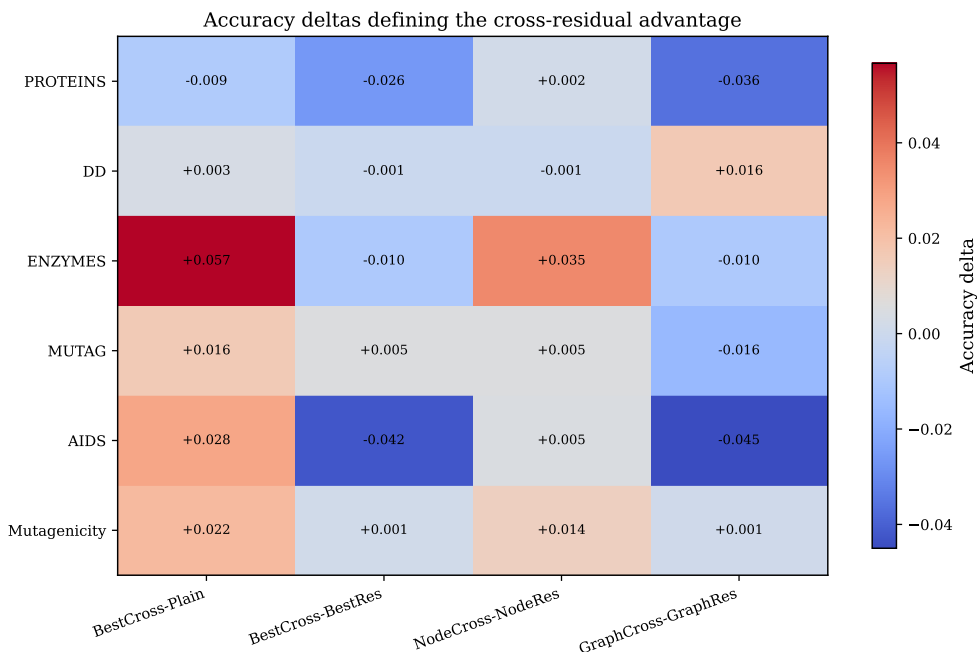


Figure 4. Accuracy deltas that define the practical value of cross-residual routing. Positive values indicate an advantage for the cross design, while negative values indicate that the corresponding plain or residual alternative remains stronger.

Figure 3 and Figure 4 make the same point from complementary angles: cross-residual routing is usually better than plain stacking, but it is not consistently stronger than the best residual alternative.

Results on the biological core datasets

The main biological benchmark results are reported in Table 3. The topic-facing interpretation is anchored in the protein-oriented core datasets. These datasets carry the main substantive weight of the paper, so the interpretation here is intentionally more detailed than in the supplementary package.

Residual reuse remains the strongest default family in the reported suite. Residual variants rank first numerically on PROTEINS, DD, ENZYMES, and AIDS. GraphResGNN leads on PROTEINS (0.7223 ± 0.0367), ENZYMES (0.3117 ± 0.0746), and AIDS (0.9160 ± 0.0121), while NodeResGNN leads on DD (0.7207 ± 0.0259). The point is not that every residual margin is large, but that residual models repeatedly remain at or near the top across the core biological tasks under the same five-fold protocol. It is important to note that many of these numerical leads are modest: paired statistical tests (reported in Table 9) show that the majority of pairwise architecture comparisons do not reach conventional significance ($p < 0.05$), a pattern that is itself informative—it indicates that architecture-level differences are smaller than split-level variation under the controlled protocol.

Cross-residual design is selectively strong, not universally dominant. Cross variants rank first numerically on MUTAG and Mutagenicity. More importantly, the best cross variant beats PlainGNN on most completed datasets. Therefore, cross routing is clearly more than a design curiosity, but the evidence does not support the stronger claim that cross universally outperforms residual reuse. This narrower reading is important for reliability because it matches both the mean results and the observed split-level variation. The few pairwise comparisons that are statistically significant (e.g., GraphResGNN > GraphCrossGNN on PROTEINS with SAGEConv, $p = 0.031$; NodeCrossGNN > GraphCrossGNN on DD with SAGEConv, $p = 0.027$) all involve graph-level cross variants losing to either residual or node-level cross models—suggesting that the graph-level cross design is the more fragile element of the family.

Node-level cross is more reliable than graph-level cross. Across the completed datasets, NodeCrossGNN outperforms NodeResGNN numerically more often than GraphCrossGNN outperforms GraphResGNN. This suggests that exchanging intermediate node-level states is the more robust cross-residual design in the present study, while graph-level cross routing is more dataset-sensitive and its advantage over graph-level residual reuse is statistically significant in the negative direction on several operator–dataset combinations (Table 9).

Plain stacking suffers from depth-induced signal collapse but is not uniformly worse. PlainGNN remains competitive on PROTEINS and DD, but it never leads a dataset in the final suite and is usually surpassed by either a residual or a cross-residual variant. On ENZYMES and AIDS under GCNConv, the gap between PlainGNN and the best residual variant is statistically significant (ENZYMES + GINConv: $p = 0.002$; AIDS + GCNConv: $p = 0.039$), consistent with the interpretation that plain stacking’s head-pattern dominance suppresses discriminative mid-range structural information on tasks where deeper propagation alone is insufficient.

Topic-focused results for PROTEINS, DD, and ENZYMES

PROTEINS. GraphResGNN leads numerically (0.7223 ± 0.0367), with an advantage of $+0.017$ over PlainGNN that does not reach statistical significance in most operator-level comparisons (e.g., GCNConv: $p = 0.375$, Cohen’s $d = +0.45$, small-to-medium effect). This suggests that tail-preserving residual routing—repeated graph-level summary reinjection—offers a modest but consistent benefit, consistent with the interpretation that GraphResGNN preserves more historical structural signal than plain stacking, without claiming a universal advantage. The best cross variant, NodeCrossGNN (0.6963 ± 0.0433), remains below GraphResGNN on this dataset.

DD. NodeResGNN (0.7207 ± 0.0259) and NodeCrossGNN (0.7198 ± 0.0178) are nearly tied ($\Delta = 0.0009$, not statistically significant across any operator). The practical interpretation is that node-level historical structural signal preservation is important on large sparse protein graphs—adding cross-branch exchange neither helps nor hurts materially. This is a null result worth reporting: it indicates that the cross-branch pathway does not introduce destructive interference on DD, but also does not provide the selective gain observed on other datasets.

ENZYMES. Cross variants become more attractive numerically. GraphCrossGNN reaches 0.3017 ± 0.0750 and NodeCrossGNN reaches 0.3000 ± 0.0580 , both clearly above PlainGNN at 0.2450 ± 0.0407 (statistically significant under GINConv: NodeCrossGNN vs PlainGNN $p = 0.009$, GraphResGNN vs PlainGNN $p = 0.002$). Even so, GraphResGNN still leads numerically (0.3117 ± 0.0746). However, the absolute accuracy across all models remains low (best model only ~ 15 percentage points above the 6-class random baseline of 16.7%), indicating that the 3-dimensional input features and 600-graph sample size create a challenging regime where representation learning is inherently noisy. The ENZYMES results should therefore be read as evidence of relative ordering rather than absolute task mastery.

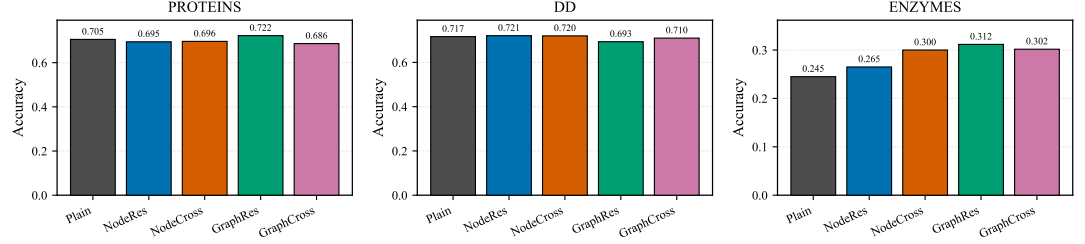


Figure 5. Focused comparison on the completed topic-facing TU datasets. The pattern is mixed rather than one-sided: graph-level residual refinement is strongest on PROTEINS and ENZYMES, while DD favors node-level reuse, with NodeCrossGNN remaining competitive but not dominant.

Mechanistic interpretation of the residual advantage

The benchmark patterns above are easier to interpret if residual reuse is viewed as a change in the effective propagation filter rather than only as an optimization trick. Under a linearized plain update, repeated message passing applies the same propagation operator $\mathbf{S}$ multiple times, so the hidden state after ℓ layers takes the form $\mathbf{H}^{(\ell)} = \mathbf{S}^\ell \mathbf{X}$. In that setting, deeper propagation increasingly suppresses non-dominant spectral components, which is the standard over-smoothing tendency. This interpretation does not replace the empirical results, but it provides the missing intuition for why the residual family may remain stronger even when deeper message passing alone does not help.

Residual reuse changes this behavior. If a residual line reuses the previous hidden state before the next propagation step, the update no longer corresponds to one pure power of $\mathbf{S}$. Instead, it produces a polynomial mixture of lower-order and higher-order propagation terms. In practical terms, the model is no longer forced to rely only on the deepest smoothing order at a fixed depth; it can preserve more mid-range structural information from earlier layers while still benefiting from deeper propagation. This interpretation aligns with the empirical fact that NodeResGNN and GraphResGNN remain stronger default references than PlainGNN across the biological core.

The dataset-specific winner pattern is also consistent with that mechanism view. On PROTEINS and ENZYMES, the strongest model is GraphResGNN, suggesting that re-injecting graph-level summaries can stabilize useful global context without forcing the model to rely exclusively on deeper propagation powers. On DD, by contrast, NodeResGNN retains the lead, which indicates that lighter node-level reuse is the more effective way to preserve discriminative intermediate structure on larger sparse protein graphs. In both cases, the key point is the same: the best residual models appear to work not simply because they are deeper, but because they reduce the dominance of the highest propagation order.

This mechanism interpretation also clarifies why cross designs are competitive but not uniformly dominant. Cross-residual exchange creates additional information routes, yet those extra routes do not automatically produce a better spectral trade-off than ordinary residual reuse. The topic-facing results show that NodeCrossGNN is often the more stable cross variant, while graph-level cross exchange is more regime-sensitive. Therefore, the main comparison should not be read as “more paths are always better.” A more defensible reading is that the benefit depends on whether the added route preserves informative structure or instead introduces unstable interactions that the dataset does not reward.

The targeted routing experiments sharpen that point further. Once the architecture comparison identifies strong residual and cross-oriented targets, the next question is how much historical information should be passed forward and how selectively that information should be filtered. The routing results show that dense learnable reuse remains strongest on the best graph-level residual line, while mild sparse routing is often strongest on representative cross-oriented targets. This is exactly the kind of behavior expected if residual pathways are useful because they regulate the effective propagation mix: some targets benefit from dense preservation of historical signal, whereas others benefit from suppressing weaker residual channels before they accumulate.

439 The ablations therefore serve a methodological purpose. They show that the reported gains are
440 not left unexplained after the benchmark stage, but can be probed through controlled changes
441 to the reuse mechanism itself.

442 The scope of this interpretation should remain disciplined. The argument above is an ex-
443 planatory mechanism, not a full theorem for the nonlinear trained models in this paper. It
444 abstracts away activations, optimization dynamics, and operator-specific details. Even so, it
445 helps connect the benchmark hierarchy to a coherent empirical story: residual reuse remains
446 the stronger default because it slows representation collapse more reliably than plain stack-
447 ing, while cross-residual interaction is useful when the additional exchange pathway preserves
448 task-relevant information rather than merely adding complexity.

449 Results on the supplementary robustness datasets

450 The supplementary robustness results are reported in Table 4. The supplementary package
451 broadens the scope beyond the biological core without changing the main conclusion. NodeCross-
452 GNN is strongest on MUTAG and Mutagenicity, whereas GraphResGNN remains strongest on
453 AIDS. This pattern preserves the paper’s main boundary claim: cross-residual interaction has
454 selective practical value, but the residual family remains the more reliable default reference
455 across the full suite.

456 Gate ablation results

457 The gate ablations are designed to test whether adaptive control of residual injection is useful
458 once the main benchmark has identified strong target configurations. On the strongest graph-
459 level residual line, AIDS + GraphResGNN + GINConv, the comparison among learnable and
460 fixed gates is the cleanest direct test of adaptive graph-level reuse. Table 6 shows that the
461 learnable gate is the strongest setting, although the fixed coefficient 0.5 remains very close. The
462 cross-gate ablation extends the same comparison to six settings where a cross architecture is
463 already locally favored. In other words, the gate study addresses a question left open by the
464 benchmark tables: if reuse helps, is the benefit coming from adaptive control or merely from
465 adding another injection path with any fixed weight?

466 Table 9 provides the complete paired-test matrix for all architecture comparisons across
467 datasets and operators. The overall pattern confirms that most numerical gaps between archi-
468 tectures are modest relative to fold-level variation: only a small fraction of comparisons reach
469 $p < 0.05$ under both the paired t -test and the Wilcoxon signed-rank test. This is consistent
470 with the paper’s bounded interpretation—differences between reuse topologies are real but con-
471 ditional, and no single architecture dominates across all settings.

472 The fold-level table is useful because it shows where that robustness comes from: the learn-
473 able gate is not just strong in the average, but remains consistently high across the five AIDS
474 folds without relying on one unusually favorable split. The fuller cross-gate table also makes
475 the mixed pattern easier to inspect target by target. The intended interpretation is therefore
476 constrained. The gate study does not need to show that learnable gating is best on every target.
477 Instead, the relevant question is whether adaptive control is generally more robust than fixed
478 injection strengths across the selected residual and cross-residual settings. Table 8 shows that
479 learnable gating is best on three of the six cross-favored targets, while fixed gates remain com-
480 petitive and can even be superior on the other three targets. This supports a robustness claim,
481 not a universal-optimum claim.

482 Residual-routing results

483 The residual-routing analysis is the main mechanism extension of the paper. Four representative
484 targets compare learnable, top-k, and sparse routing modes under the same five-fold protocol.
485 The clearest overall signal is that there is no universal winner. Table 10 shows that learnable
486 remains best on the strongest graph-level residual line (AIDS + GraphResGNN + GINConv),
487 but sparse routing is strongest on three of the four representative targets, with sparse_0p05
488 emerging as the most consistent sparse setting. The table keeps the exact parameter-level
489 comparison in the main text, so the reader can see directly that the sparse advantage is not a
490 vague family-level trend but a repeated target-level outcome. This substantially reduces the risk

Table 6. AIDS gate ablation on GraphResGNN + GINConv. Numbers are five-fold mean accuracy $\pm$ standard deviation. Bold: best per column.

Gate setting	Mean accuracy
Learnable	0.9620 $\pm$ 0.0089
Fixed 0.0	0.9150 $\pm$ 0.0138
Fixed 0.5	0.9605 $\pm$ 0.0144
Fixed 1.0	0.9480 $\pm$ 0.0300

Table 7. Fold-level AIDS gate ablation on AIDS + GraphResGNN + GINConv. Bold: best per column.

Gate setting	Fold 0	Fold 1	Fold 2	Fold 3	Fold 4	Mean $\pm$ Std
Learnable	0.9500	0.9700	0.9700	0.9675	0.9525	0.9620 $\pm$ 0.0089
Fixed 0.0	0.9225	0.9025	0.9125	0.9000	0.9375	0.9150 $\pm$ 0.0138
Fixed 0.5	0.9325	0.9625	0.9675	0.9675	0.9725	0.9605 $\pm$ 0.0144
Fixed 1.0	0.8925	0.9450	0.9700	0.9775	0.9550	0.9480 $\pm$ 0.0300

of over-interpreting a single average number, because the comparison is repeated across several targets and routing settings rather than inferred from one isolated benchmark cell.

This pattern supports a more precise conclusion than the architecture-only benchmark alone. The important question is not only whether historical information is reused, but also how aggressively that information is filtered before reuse. Table 11 compresses the result by routing family: the sparse family is strongest on three of the four representative targets, while the learnable family remains strongest on the best graph-level residual line.

Residual-parameter sweep results

The additional routing settings `topk_0p75` and `sparse_0p1` refine the previous result rather than changing it. Table 10 shows that the strongest top-k ratio varies by target, which indicates that top-k routing is not governed by one globally optimal retention level. The sparse sweep also shows a clear failure mode: aggressive sparsity can damage otherwise strong residual lines, especially on the strongest graph-level residual target, where `sparse_0p1` drops to 0.8315 ± 0.0549 .

Exploratory sensitivity analysis

Auxiliary sensitivity checks over depth, dropout, and learning rate on PROTEINS, DD, and ENZYMES do not materially change the main conclusion. On PROTEINS and DD, moderate tuning changes affect the gap between the two cross variants, but they do not overturn the broader conclusion that residual baselines remain the stronger default reference. On ENZYMES, the validation signal is visibly noisier, and promising local settings did not reproduce the original full-suite cross results under the final 5-fold average. These checks are reported for transparency, but they should not be read as evidence at the same level as the repeated-split benchmark and targeted ablations.

DISCUSSION

The main empirical message is now more precise than in the earlier draft. Cross-residual routing is usually beneficial relative to plain stacking, but the gain over residual baselines is conditional and dataset-dependent. The present evidence therefore supports CR-GNN as a meaningful biomolecular graph-classification design space rather than as a universally dominant model family. This framing is important for interpreting both the strengths and the limits of the paper: the contribution lies in identifying a reproducible hierarchy of reuse mechanisms, not in claiming a new universal best model.

Table 8. Cross-gate ablation on six cross-favored targets. Numbers are five-fold mean accuracy $\pm$ standard deviation. Bold: best per row.

Dataset	Target model/operator	Learnable	Fixed 0.0	Fixed 0.5	Fixed 1.0
AIDS	NodeCrossGNN + GATConv	0.9045 $\pm$ 0.0165	0.9075 $\pm$ 0.0065	0.9195 $\pm$ 0.0114	0.9105 $\pm$ 0.0241
DD	NodeCrossGNN + GATConv	0.7173 $\pm$ 0.0271	0.7053 $\pm$ 0.0361	0.7156 $\pm$ 0.0225	0.7147 $\pm$ 0.0288
DD	NodeCrossGNN + GINConv	0.7071 $\pm$ 0.0306	0.7012 $\pm$ 0.0198	0.7232 $\pm$ 0.0207	0.7113 $\pm$ 0.0277
ENZYMES	NodeCrossGNN + GATConv	0.2800 $\pm$ 0.0629	0.2850 $\pm$ 0.0627	0.2550 $\pm$ 0.0674	0.2650 $\pm$ 0.0690
MUTAG	GraphCrossGNN + GATConv	0.7558 $\pm$ 0.0495	0.7451 $\pm$ 0.0407	0.7450 $\pm$ 0.0416	0.7504 $\pm$ 0.0779
Mutagenicity	NodeCrossGNN + GATConv	0.8114 $\pm$ 0.0110	0.8033 $\pm$ 0.0192	0.8010 $\pm$ 0.0130	0.8036 $\pm$ 0.0078

Cross-residual gains are conditional

The benchmark results do not support the strongest possible claim that cross-residual interaction should replace residual reuse as the default choice. Instead, they support a narrower statement: cross-residual design is often better than plain stacking and can provide selective gains on specific datasets—particularly where rare discriminative motif preservation matters, as on MUTAG and Mutagenicity—but ordinary residual reuse remains the stronger default family across the full biomolecular package. This narrower conclusion is more convincing because it is aligned with the five-fold mean $\pm$ standard deviation reporting and the pairwise statistical tests (Table 9) rather than with isolated best-case gains.

The dataset-level pattern is informative. On MUTAG, where NodeCrossGNN leads, the task involves small molecular graphs (avg. 18 nodes) where local structural motifs (e.g., specific chemical substructures) carry strong discriminative signal. Cross-branch exchange may help here by preserving rare motifs that a single branch tends to suppress under repeated propagation—a form of rare discriminative motif preservation through complementary parallel pathways. On Mutagenicity, the larger sample size (4,337 graphs) reduces split-level variance, making the NodeCrossGNN advantage (+0.014 over NodeResGNN) more stable though still statistically modest. On AIDS, by contrast, GraphResGNN’s strong numerical lead (0.9160 vs. 0.8735 for NodeCrossGNN) suggests that when graph-level context dominates, tail-preserving residual routing through sequential graph-conditioned blocks is the more effective strategy. This conditional pattern—cross helps when local motifs are rare and discriminative, residual helps when global context dominates—is the paper’s central empirical observation.

Node-level and graph-level reuse behave differently

The distinction between node-level and graph-level information reuse is also practically important. Node-level cross exchange is the more stable cross-oriented design, whereas graph-level cross routing is more sensitive to dataset regime and does not consistently match the strongest graph-level residual lines. This suggests that the location of reuse is at least as important as the existence of an additional reuse path. Importantly, GraphResGNN and GraphCrossGNN carry more parameters than their node-level counterparts (e.g., GraphResGNN $\approx$ 51k vs. NodeResGNN $\approx$ 23k parameters on PROTEINS, Table 5), because chaining multiple graph-conditioned blocks increases the total parameter budget. The fact that GraphResGNN’s numerical lead does not scale uniformly with its parameter advantage—it ranks first on PROTEINS and ENZYMES but is surpassed by lighter node-level models on DD and MUTAG—suggests that the information-flow topology, rather than parameter count alone, is the more important determinant of relative performance.

Why ENZYMES remains difficult for all architectures

A notable result is that all models perform poorly on ENZYMES in absolute terms: the best model (GraphResGNN, 0.3117 $\pm$ 0.0746) is only $\sim$ 15 percentage points above the 6-class random baseline (16.7%). Several factors likely contribute. First, ENZYMES has only 600 graphs with 3-dimensional input features, making representation learning challenging and the inner validation split (10%) noisy. Second, the 6-class structure creates a harder decision boundary than the binary tasks that dominate the remaining benchmark suite. Third, the observed fold-level standard deviation ($\pm$ 0.075) is the largest in the biological core, indicating high split sensitivity. This does not invalidate the relative ranking—the residual family still leads—but it bounds the

565 strength of any architecture-level conclusion drawn from ENZYMES alone. The ENZYMES
566 results are best interpreted as evidence that tail-preserving residual routing provides relative
567 stability even under noisy, low-data regimes, rather than as evidence that any architecture
568 solves the task well.

569 Residual routing is part of the mechanism story
570 The residual-routing ablations add an important qualification to the main architecture com-
571 parison. The question is not only whether residual information should be reused, but also
572 whether it should be reused densely, sparsified, or filtered by top-k selection—a question of se-
573 lective residual filtering rather than binary reuse. The current results show that routing strategy
574 is target-dependent: learnable dense routing remains strongest on the best graph-level residual
575 line where tail-preserving residual routing benefits from rich historical context, while mild sparse
576 routing is frequently advantageous on cross-oriented settings where suppressing weak residual
577 channels reduces head-pattern dominance and prevents unfiltered cross-branch signal accumu-
578 lation. This shifts the paper away from a simple architecture ranking exercise and toward a
579 mechanism study in which architecture and routing are interpreted together.

580 Evidence hierarchy and study limitations
581 The strongest evidence in the paper comes from the five-fold LATEST benchmark and the five-
582 fold targeted ablations. By contrast, the older single-fold sensitivity scans are useful only for op-
583 timization interpretation. They should not be treated as the same class of quantitative evidence.
584 A second limitation is statistical power: the paired significance tests (Table 9) show that most
585 architecture-level numerical gaps do not reach $p < 0.05$ under five-fold evaluation. With only five
586 paired observations per dataset-operator combination, the tests have limited power to detect
587 small-to-medium effects (Cohen’s $d < 0.5$). This does not undermine the paper’s bounded con-
588 clusions, but it means that the claim “X is stronger than Y” should be understood as a numerical
589 ordering observed under controlled conditions rather than a statement of statistical dominance.
590 Future work with more folds or repeated random subsampling would provide sharper pairwise
591 comparisons. A third limitation is baseline scope. The benchmark includes representative resid-
592 ual, propagation, and aggregation baselines, but it is not intended as an exhaustive leaderboard
593 against every recent graph-classification method. The paper is instead designed to support a
594 controlled comparison of reuse topologies under a stable protocol. A fourth limitation is dataset
595 scale and task scope: although the datasets are biomolecular or molecular and the biological
596 core is meaningful, they are still medium-scale benchmark datasets for graph classification. The
597 study therefore supports conclusions about controlled graph-classification behavior, not about
598 large-scale scalability, direct biological discovery, omics integration, or task-specific biological
599 interpretation.

600 Future directions
601 The next research step is to test the same information-flow ideas with richer biological inputs,
602 such as sequence-derived embeddings, ontology annotations, or expression-related signals. An-
603 other important extension is to evaluate the same controlled reuse questions on larger graph
604 benchmarks, so that the current mechanism-oriented conclusions can be tested beyond the
605 present medium-scale setting. In parallel, the residual-routing perspective introduced here could
606 be extended to broader biomolecular graph tasks where graph topology must be combined with
607 stronger domain supervision rather than benchmark-only evaluation.

608 CONCLUSIONS

609 This paper studies CR-GNN as a controlled biomolecular graph-classification framework rather
610 than as a claim of universal model superiority. The final evidence supports a narrower but
611 stronger conclusion: cross-residual interaction is useful, but residual variants remain the strongest
612 default family in the reported benchmark suite, and the gains are conditional on both dataset
613 regime and routing strategy.

614 The main contribution is therefore not a universally dominant architecture, but a controlled
615 comparison of information-flow mechanisms and residual-routing strategies that reveals when

and how residual routing helps. Across the biological core, GraphResGNN leads numerically on PROTEINS and ENZYMES through tail-preserving residual routing—repeated reinjection of graph-level summaries—while NodeResGNN leads on DD through node-level historical structural signal preservation. Cross-residual variants provide selective gains on MUTAG and Mutagenicity, where rare discriminative motif preservation via complementary parallel pathways matters most, but they do not replace residual baselines as the main point of reference. Importantly, most numerical gaps between architectures are modest relative to fold-level variation; only a subset of comparisons reach $p < 0.05$ under paired statistical tests, reinforcing the paper’s bounded interpretation.

The routing analysis sharpens that result further. Learnable dense routing remains strongest on the best graph-level residual line, while mild sparse routing (selective residual filtering) is frequently effective on cross-oriented settings. This shows that the practical question is not only whether an additional residual path exists, but how historical information is filtered before reuse. The targeted routing results therefore help explain why the benchmark hierarchy is mixed rather than one-sided: the benefit of an extra reuse path depends on whether it preserves task-relevant structure or introduces head-pattern dominance through unfiltered signal accumulation.

The scope of this conclusion should remain disciplined. The strongest claims in the paper are supported by the stratified five-fold benchmark and the targeted five-fold ablations, whereas the older sensitivity scans are only supportive. The statistical evidence (Table 9) is informative precisely because it confirms that architecture-level effects are modest under the controlled protocol—a pattern that aligns with the paper’s mechanism-oriented rather than leaderboard-oriented framing. Likewise, the present study addresses controlled graph-classification behavior on medium-scale biomolecular and molecular benchmarks; it does not by itself establish large-scale scalability or direct biological interpretability.

Taken together, these results position CR-GNN as a practical biomolecular graph-classification design space and provide a methodological basis for future work on richer biological inputs, larger benchmarks, and graph-learning settings where residual reuse must be coordinated with stronger domain supervision.

ACKNOWLEDGMENTS

The authors thank the open-source graph learning community and the maintainers of the benchmark resources used in this study. They also thank the reviewers and editors for their time and feedback.

REFERENCES

- Uri Alon and Eran Yahav. On the bottleneck of graph neural networks. In International Conference on Learning Representations (ICLR), 2021.
- Xavier Bresson and Thomas Laurent. Residual gated graph convnets. arXiv preprint arXiv:1711.07553, 2017.
- Tianle Cai, Jiacheng Luo, Sheng Wang, Kai Zhang, Yu Tian, Liwei Yang, Zekun Li, and Kilian Weinberger. Graphnorm: A principled approach to accelerating graph neural network training. In International Conference on Machine Learning, pages 1277–1287, 2021.
- Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, and Yaliang Li. Simple and deep graph convolutional networks. In International Conference on Machine Learning (ICML), pages 1725–1735, 2020a.
- Yuning Chen, Xihao Wei, Pan Xu, Xinwen Wang, Ping Luo, Zhiyuan Li, and Jian Tang. Revisiting over-smoothing in deep gcn. arXiv preprint arXiv:2003.13663, 2020b.
- Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with pytorch geometric. arXiv preprint arXiv:1903.02428, 2019.
- Hongyang Gao and Shuiwang Ji. Graph u-net. arXiv preprint arXiv:1905.05178, 2019.

664 Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural
665 message passing for quantum chemistry. In International Conference on Machine Learning,
666 pages 1263–1272, 2017.

667 William L Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large
668 graphs. In Advances in neural information processing systems, pages 1024–1034, 2017.

669 Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image
670 recognition. In Proceedings of the IEEE conference on computer vision and pattern recogni-
671 tion, pages 770–778, 2016.

672 Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional net-
673 works. arXiv preprint arXiv:1609.02907, 2016.

674 Johannes Klicpera, Alan Bojchevski, and Stephan Günnemann. Combining label propagation
675 and simple models out-performs graph neural networks. arXiv preprint arXiv:1810.05997,
676 2018.

677 Guohao Li, Elias Muller, Abbas Thabet, and Bernard Ghanem. Deeper insights into graph
678 convolutional networks for semi-supervised learning. arXiv preprint arXiv:1809.02584, 2018.

679 Christopher Morris, Roger Wattenhofer, and Kristian Kersting. Tudataset: A collection of
680 benchmark datasets for learning with graphs. arXiv preprint arXiv:2007.08663, 2020.

681 Kenta Oono and Taiji Suzuki. Simple and deep graph convolutional networks. arXiv preprint
682 arXiv:2006.13604, 2020.

683 Yu Rong, Wenbing Huang, Tingyang Xu, and Junzhou Huang. Dropedge: Towards deep graph
684 convolutional networks on node classification. arXiv preprint arXiv:1907.10903, 2019.

685 Jake Topping, Francesco Di Giovanni, Benjamin Chamberlain, Michael Bronstein, Douwe Ven-
686 drig, and Pietro Lio. Understanding over-squashing and bottlenecks on graphs via curvature.
687 arXiv preprint arXiv:2111.14522, 2021.

688 Zonghan Wu, Shirui Pan, Fengwen Chen, Guoding Long, Cheng Zhang, and Sheng Yu Zhang.
689 A comprehensive survey on graph neural networks. IEEE Transactions on Neural Networks
690 and Learning Systems, 32(1):4–24, 2021.

691 Keyulu Xu, Chengtao Li, Yonglong Tian, Xiao Du, Jure Leskovec, and Stefanie Jegelka. Repre-
692 sentation learning on graphs with jumping knowledge networks. In International Conference
693 on Machine Learning, pages 5449–5458, 2018.

694 Rex Ying, Jiaxuan You, Christopher Morris, Xiang Ren, William L Hamilton, and Jure Leskovec.
695 Hierarchical graph representation learning with differentiable pooling. In Advances in neural
696 information processing systems, pages 4800–4810, 2018.

697 Lingxiao Zhao and Leman Akoglu. Pairnorm: Tackling over-smoothing in gns. arXiv preprint
698 arXiv:2009.10698, 2020.

699 Jie Zhou, Ganqu Cui, Zhengyan Zhang, Cheng Yang, Zhaocheng Liu, Zhongyuan Wang, Peng
700 Li, Ming Sun, Yu Shi, et al. Graph neural networks: A review of methods and applications.
701 AI Open, 1:57–81, 2020.

Table 9. Paired t -tests for architecture comparisons (transposed). Rows: dataset $\times$ comparison. Columns: operator. Cells: Δ (Cohen’s d , p). Bold: $p < 0.05$ ($n=5$ folds).

Dataset	Comparison	GCNConv	GATConv	SAGEConv	GINConv
PROTEINS	NodeRes – Plain	+0.012 (+0.55, 0.286)	+0.000 (+0.00, 0.999)	+0.006 (+0.31, 0.524)	+0.012 (+0.00, 0.286)
	NodeCross – Plain	-0.008 (-0.28, 0.571)	-0.011 (-0.22, 0.647)	-0.002 (-0.07, 0.881)	-0.008 (-0.28, 0.571)
	GraphRes – Plain	+0.004 (+0.45, 0.375)	+0.015 (+0.48, 0.345)	+0.030 (+1.69, 0.019)	+0.040 (+1.69, 0.019)
	GraphCross – Plain	-0.001 (-0.02, 0.961)	-0.008 (-0.24, 0.623)	+0.004 (+0.24, 0.622)	+0.004 (+0.24, 0.622)
	NodeCross – NodeRes	-0.020 (-0.53, 0.302)	-0.011 (-0.19, 0.690)	-0.008 (-0.72, 0.181)	-0.020 (-0.53, 0.302)
	GraphCross – GraphRes	-0.005 (-0.15, 0.751)	-0.023 (-2.18, 0.008)	-0.026 (-1.47, 0.031)	-0.036 (-2.18, 0.008)
	GraphRes – NodeRes	-0.007 (-0.26, 0.588)	+0.015 (+0.37, 0.459)	+0.023 (+0.75, 0.171)	+0.028 (+0.75, 0.171)
	NodeCross – GraphCross	-0.007 (-0.49, 0.337)	-0.003 (-0.10, 0.838)	-0.005 (-0.21, 0.659)	-0.012 (-0.49, 0.337)
DD	NodeRes – Plain	+0.012 (+0.39, 0.429)	+0.002 (+0.24, 0.624)	+0.016 (+0.48, 0.347)	+0.001 (+0.39, 0.429)
	NodeCross – Plain	+0.019 (+0.55, 0.286)	+0.035 (+0.90, 0.115)	+0.023 (+0.87, 0.124)	+0.025 (+0.55, 0.286)
	GraphRes – Plain	+0.030 (+0.64, 0.223)	+0.020 (+0.50, 0.326)	+0.021 (+0.91, 0.112)	+0.002 (+0.64, 0.223)
	GraphCross – Plain	+0.024 (+0.87, 0.123)	+0.019 (+0.46, 0.357)	+0.004 (+0.19, 0.699)	+0.008 (+0.87, 0.123)
	NodeCross – NodeRes	+0.008 (+0.43, 0.395)	+0.033 (+0.88, 0.121)	+0.007 (+0.64, 0.226)	+0.024 (+0.43, 0.395)
	GraphCross – GraphRes	-0.007 (-0.23, 0.637)	-0.002 (-0.11, 0.815)	-0.017 (-2.82, 0.003)	+0.006 (-0.23, 0.637)
	GraphRes – NodeRes	+0.019 (+0.64, 0.228)	+0.019 (+0.46, 0.359)	+0.005 (+0.24, 0.624)	+0.001 (+0.64, 0.228)
	NodeCross – GraphCross	-0.004 (-0.31, 0.526)	+0.016 (+1.07, 0.075)	+0.019 (+1.53, 0.027)	+0.017 (-0.31, 0.526)
ENZYMES	NodeRes – Plain	-0.000 (-0.00, 1.000)	+0.005 (+0.19, 0.697)	-0.017 (-0.94, 0.103)	+0.030 (-0.00, 1.000)
	NodeCross – Plain	+0.003 (+0.09, 0.847)	+0.032 (+0.45, 0.369)	+0.007 (+0.24, 0.614)	+0.032 (+0.09, 0.847)
	GraphRes – Plain	+0.065 (+0.60, 0.248)	+0.030 (+0.83, 0.137)	+0.082 (+0.88, 0.121)	+0.077 (+0.60, 0.248)
	GraphCross – Plain	+0.017 (+0.31, 0.526)	-0.000 (-0.00, 1.000)	+0.005 (+0.21, 0.666)	+0.032 (+0.31, 0.526)
	NodeCross – NodeRes	+0.003 (+0.07, 0.883)	+0.027 (+0.47, 0.350)	+0.023 (+1.29, 0.045)	+0.002 (+0.07, 0.883)
	GraphCross – GraphRes	-0.048 (-0.59, 0.261)	-0.030 (-1.56, 0.025)	-0.077 (-0.93, 0.106)	-0.045 (-0.59, 0.261)
	GraphRes – NodeRes	+0.065 (+0.56, 0.280)	+0.025 (+0.68, 0.203)	+0.098 (+1.15, 0.062)	+0.047 (+0.56, 0.280)
	NodeCross – GraphCross	-0.013 (-0.33, 0.500)	+0.032 (+0.61, 0.244)	+0.002 (+0.04, 0.928)	-0.000 (-0.33, 0.500)
MUTAG	NodeRes – Plain	+0.011 (+0.73, 0.178)	+0.011 (+0.15, 0.754)	-0.006 (-0.25, 0.609)	-0.005 (-0.73, 0.178)
	NodeCross – Plain	+0.006 (+0.25, 0.609)	+0.027 (+0.30, 0.543)	-0.027 (-0.70, 0.190)	-0.016 (-0.25, 0.609)
	GraphRes – Plain	-0.011 (-0.19, 0.692)	+0.006 (+0.05, 0.909)	-0.032 (-1.08, 0.073)	-0.010 (-0.19, 0.692)
	GraphCross – Plain	+0.005 (+0.45, 0.374)	+0.038 (+0.52, 0.312)	-0.005 (-0.11, 0.823)	-0.016 (-0.45, 0.374)
	NodeCross – NodeRes	-0.005 (-0.45, 0.374)	+0.016 (+0.67, 0.208)	-0.021 (-0.61, 0.246)	-0.011 (-0.45, 0.374)
	GraphCross – GraphRes	+0.016 (+0.33, 0.501)	+0.032 (+0.80, 0.149)	+0.027 (+0.54, 0.293)	-0.006 (+0.33, 0.501)
	GraphRes – NodeRes	-0.021 (-0.39, 0.432)	-0.005 (-0.15, 0.760)	-0.027 (-1.39, 0.036)	-0.005 (-0.39, 0.432)
	NodeCross – GraphCross	+0.000 (+0.01, 0.987)	-0.011 (-0.44, 0.381)	-0.021 (-0.45, 0.376)	+0.000 (+0.01, 0.987)
AIDS	NodeRes – Plain	+0.032 (+0.56, 0.276)	-0.016 (-0.36, 0.460)	-0.005 (-0.25, 0.607)	+0.002 (+0.56, 0.276)
	NodeCross – Plain	+0.056 (+1.35, 0.039)	+0.017 (+0.99, 0.092)	+0.000 (+0.00, 1.000)	+0.037 (+1.35, 0.039)
	GraphRes – Plain	+0.080 (+1.36, 0.039)	-0.006 (-0.15, 0.752)	+0.029 (+0.54, 0.295)	+0.047 (+1.36, 0.039)
	GraphCross – Plain	+0.045 (+0.81, 0.146)	-0.006 (-0.54, 0.290)	-0.010 (-0.48, 0.341)	-0.007 (+0.81, 0.146)
	NodeCross – NodeRes	+0.024 (+0.89, 0.118)	+0.032 (+0.69, 0.196)	+0.005 (+0.60, 0.249)	+0.035 (+0.89, 0.118)
	GraphCross – GraphRes	-0.035 (-0.92, 0.109)	-0.000 (-0.01, 0.980)	-0.040 (-0.91, 0.113)	-0.054 (-0.92, 0.109)
	GraphRes – NodeRes	+0.048 (+1.39, 0.036)	+0.010 (+0.17, 0.724)	+0.034 (+0.62, 0.235)	+0.045 (+1.39, 0.036)
	NodeCross – GraphCross	+0.011 (+0.56, 0.282)	+0.023 (+1.73, 0.018)	+0.010 (+0.28, 0.562)	+0.044 (+0.56, 0.282)
Mutagenicity	NodeRes – Plain	+0.004 (+0.26, 0.597)	+0.011 (+0.95, 0.101)	+0.014 (+0.93, 0.106)	+0.013 (+0.26, 0.597)
	NodeCross – Plain	+0.011 (+0.68, 0.201)	+0.020 (+1.05, 0.079)	+0.013 (+0.86, 0.127)	+0.017 (+0.68, 0.201)
	GraphRes – Plain	+0.019 (+1.24, 0.050)	+0.008 (+0.57, 0.273)	+0.012 (+0.62, 0.237)	+0.018 (+1.24, 0.050)
	GraphCross – Plain	+0.016 (+0.85, 0.131)	+0.009 (+0.46, 0.366)	+0.012 (+0.49, 0.333)	+0.010 (+0.85, 0.131)
	NodeCross – NodeRes	+0.006 (+0.26, 0.588)	+0.009 (+0.69, 0.195)	-0.001 (-0.13, 0.787)	+0.003 (+0.26, 0.588)
	GraphCross – GraphRes	-0.003 (-0.30, 0.545)	+0.002 (+0.08, 0.874)	+0.000 (+0.00, 0.999)	-0.008 (-0.30, 0.545)
	GraphRes – NodeRes	+0.015 (+0.87, 0.125)	-0.003 (-0.27, 0.583)	-0.003 (-0.18, 0.714)	+0.005 (+0.87, 0.125)
	NodeCross – GraphCross	-0.005 (-0.35, 0.476)	+0.010 (+0.50, 0.324)	+0.001 (+0.10, 0.832)	+0.007 (-0.35, 0.476)

Table 10. Residual-routing and parameter-sweep summary on four representative targets. Numbers are five-fold mean accuracy $\pm$ standard deviation.

Target	Learn.	T-k 0.25	T-k 0.5	T-k 0.75	Sp. 0
AIDS + GraphResGNN + GINConv	0.9500 $\pm$ 0.0087	0.9075 $\pm$ 0.0237	0.9330 $\pm$ 0.0251	0.9240 $\pm$ 0.0287	0.9470 $\pm$ 0.0087
PROTEINS + GraphResGNN + SAGEConv	0.7178 $\pm$ 0.0411	0.7241 $\pm$ 0.0407	0.7151 $\pm$ 0.0397	0.7214 $\pm$ 0.0318	0.7160 $\pm$ 0.0411
AIDS + NodeCrossGNN + GATConv	0.8970 $\pm$ 0.0491	0.9160 $\pm$ 0.0128	0.8960 $\pm$ 0.0483	0.8960 $\pm$ 0.0489	0.9060 $\pm$ 0.0491
DD + NodeCrossGNN + GATConv	0.7080 $\pm$ 0.0172	0.7062 $\pm$ 0.0249	0.7029 $\pm$ 0.0160	0.7164 $\pm$ 0.0248	0.7147 $\pm$ 0.0172

Table 11. Family-wise routing winners.

Target	Best learnable	Best top-k	Best sparse	Overall winner
AIDS + GraphResGNN + GINConv	0.9500	0.9330	0.9470	Learnable
PROTEINS + GraphResGNN + SAGEConv	0.7178	0.7241	0.7259	Sparse
AIDS + NodeCrossGNN + GATConv	0.8970	0.9160	0.9180	Sparse
DD + NodeCrossGNN + GATConv	0.7080	0.7164	0.7181	Sparse